

Templates Vue.js + Pinia - IPM

Baseado nas Cheat Sheets - Padrões Corretos

ÍNDICE

1. [Componentes Vue Básicos](#)
 2. [Reatividade \(ref, reactive, computed\)](#)
 3. [Diretivas de Template](#)
 4. [Lifecycle Hooks](#)
 5. [Comunicação entre Componentes](#)
 6. [Pinia Stores](#)
 7. [Vue Router](#)
 8. [Fetch API e Async](#)
 9. [Formulários e v-model](#)
 10. [Slots](#)
 11. [Web Components](#)
-

1. COMPONENTES VUE BÁSICOS

Template Básico - Composition API (RECOMENDADO)

vue

```
<template>
  <div>
    <h1>{{ title }}</h1>
    <p>Count: {{ count }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>
```

```
<script setup>
import { ref } from 'vue';

// Propiedades reativas
const title = ref('My Component');
const count = ref(0);
```

```
// Métodos
function increment() {
  count.value++;
}
</script>
```

```
<style scoped>
h1 {
  color: #3498db;
}
</style>
```

Template Options API (Alternativa)

vue

```
<template>
  <div>
    <h1>{{ title }}</h1>
    <p>Count: {{ count }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>

<script>
export default {
  name: 'MyComponent',

  data() {
    return {
      title: 'My Component',
      count: 0
    };
  },

  methods: {
    increment() {
      this.count++;
    }
  }
}
</script>

<style scoped>
h1 {
  color: #3498db;
}
</style>
```

2. REATIVIDADE

ref() - Para Valores Primitivos

vue

```
<script setup>
import { ref } from 'vue';

// ✓ CORRETO - ref para primitivos
const count = ref(0);
const name = ref('João');
const isActive = ref(true);

// Modificar (usa .value no SCRIPT)
count.value++;
name.value = 'Maria';

// ✗ ERRADO - reactive para primitivos
// const count = reactive(0); // NÃO FUNCIONA!
</script>

<template>
  <!-- No template NÃO usa .value -->
  <p>{{ count }}</p>
  <p>{{ name }}</p>
</template>
```

reactive() - Para Objetos e Arrays

vue

```
<script setup>
import { reactive } from 'vue';

// ✓ CORRETO - reactive para objetos
const state = reactive({
  name: 'João',
  age: 25,
  items: []
});

// Modificar (sem .value)
state.name = 'Maria';
state.items.push('item');
</script>

<template>
  <p>{{ state.name }} - {{ state.age }}</p>
</template>
```

computed() - Propriedades Calculadas

vue

```
<script setup>
import { ref, computed } from 'vue';

const count = ref(5);

// ✓ CORRETO - computed é cached e atualiza automaticamente
const doubled = computed(() => count.value * 2);
const isEven = computed(() => count.value % 2 === 0);

// Computed com getter e setter
const fullName = computed({
  get() {
    return `${firstName.value} ${lastName.value}`;
  },
  set(newValue) {
    [firstName.value, lastName.value] = newValue.split(' ');
  }
});
</script>

<template>
  <p>Count: {{ count }}</p>
  <p>Doubled: {{ doubled }}</p>
  <p>Is Even: {{ isEven }}</p>
</template>
```

watch() e watchEffect()

vue

```
<script setup>
import { ref, watch, watchEffect } from 'vue';

const count = ref(0);
const name = ref('João');

// watch - observar mudanças específicas
// ⚠️ NÃO executa imediatamente por defeito
watch(count, (newVal, oldVal) => {
  console.log(`Count mudou de ${oldVal} para ${newVal}`);
});

// watch com immediate: true
watch(count, (newVal) => {
  console.log('Executado imediatamente e depois de cada mudança');
}, { immediate: true });

// watch múltiplas fontes
watch([count, name], ([newCount, newName], [oldCount, oldName]) => {
  console.log('Algum valor mudou');
});

// watchEffect - tracking automático, executa IMEDIATAMENTE
watchEffect(() => {
  console.log(`Count é ${count.value}`);
  // Automaticamente re-executa quando count muda
});
</script>
```

3. DIRETIVAS DE TEMPLATE

v-model - Two-Way Binding

vue

```
<template>
  <!-- Input de texto -->
  <input v-model="text" placeholder="Digite algo" />

  <!-- Checkbox -->
  <input type="checkbox" v-model="isChecked" />

  <!-- Radio buttons -->
  <input type="radio" value="A" v-model="selected" />
  <input type="radio" value="B" v-model="selected" />

  <!-- Select -->
  <select v-model="selectedOption">
    <option value="opt1">Opção 1</option>
    <option value="opt2">Opção 2</option>
  </select>

  <!-- Textarea -->
  <textarea v-model="message"></textarea>
</template>

<script setup>
import { ref } from 'vue';

const text = ref("");
const isChecked = ref(false);
const selected = ref("");
const selectedOption = ref("");
const message = ref("");
</script>
```

v-for - Iteração

vue

```
<template>
  <!-- ✓ CORRETO - Iterar array com :key ÚNICO -->
  <ul>
    <li v-for="item in items" :key="item.id">
      {{ item.name }}
    </li>
  </ul>

  <!-- Com índice -->
  <ul>
    <li v-for="(item, index) in items" :key="item.id">
      {{ index + 1 }}. {{ item.name }}
    </li>
  </ul>

  <!-- Iterar objeto -->
  <ul>
    <li v-for="(value, key) in user" :key="key">
      {{ key }}: {{ value }}
    </li>
  </ul>

  <!-- Com índice em objeto -->
  <ul>
    <li v-for="(value, key, index) in user" :key="key">
      {{ index }}. {{ key }}: {{ value }}
    </li>
  </ul>

  <!-- ✗ ERRADO - usar index como key em listas que mudam -->
  <!-- <li v-for="(item, index) in items" :key="index"> -->
</template>
```

```
<script setup>
import { ref } from 'vue';

const items = ref([
  { id: 1, name: 'Item A' },
  { id: 2, name: 'Item B' },
  { id: 3, name: 'Item C' }
]);

const user = ref({
  name: 'João',
  age: 25,
  email: 'joao@example.com'
});
```



```
</script>
```

v-if / v-else-if / v-else e v-show

vue

```
<template>
  <!-- v-if - Remove do DOM -->
  <div v-if="count > 10">
    Maior que 10
  </div>
  <div v-else-if="count > 5">
    Entre 5 e 10
  </div>
  <div v-else>
    5 ou menos
  </div>

  <!-- v-show - Toggle CSS display (melhor para toggle frequente) -->
  <div v-show="isVisible">
    Apenas escondido com CSS
  </div>
</template>

<script setup>
import { ref } from 'vue';

const count = ref(7);
const isVisible = ref(true);
</script>
```

Event Handlers (@click, @input, etc)

vue

```
<template>
  <!-- @click = v-on:click -->
  <button @click="handleClick">Click me</button>

  <!-- Com parâmetros -->
  <button @click="increment(5)">Add 5</button>

  <!-- Com event object -->
  <button @click="handleEvent($event)">Get Event</button>

  <!-- Modificadores -->
  <form @submit.prevent="handleSubmit">
    <button type="submit">Submit</button>
  </form>

  <input @keyup.enter="handleEnter" />

  <!-- Múltiplos eventos -->
  <button
    @click="handleClick"
    @mouseenter="handleHover"
    @mouseleave="handleLeave"
  >
    Hover me
  </button>
</template>

<script setup>
function handleClick() {
  console.log('Clicked!');
}

function increment(amount) {
  count.value += amount;
}

function handleEvent(event) {
  console.log(event.target);
}

function handleSubmit() {
  console.log('Form submitted');
}
</script>
```

:class e :style - Binding Dinâmico

vue

```
<template>
  <!-- Classes dinâmicas - objeto -->
  <div :class="{ active: isActive, 'text-danger': hasError }">
    Class object syntax
  </div>

  <!-- Classes dinâmicas - array -->
  <div :class="[isActive ? 'active' : '', errorClass]">
    Class array syntax
  </div>

  <!-- Classes dinâmicas - combinado -->
  <div :class="['base-class', { active: isActive }]">
    Combined syntax
  </div>

  <!-- Styles dinâmicos - objeto -->
  <div :style="{ color: textColor, fontSize: fontSize + 'px' }">
    Style object syntax
  </div>

  <!-- Styles dinâmicos - array -->
  <div :style="[baseStyles, overrideStyles]">
    Style array syntax
  </div>
</template>

<script setup>
import { ref } from 'vue';

const isActive = ref(true);
const hasError = ref(false);
const errorClass = ref('error');

const textColor = ref('red');
const fontSize = ref(16);

const baseStyles = {
  color: 'blue',
  fontSize: '14px'
};

const overrideStyles = {
  fontWeight: 'bold'
};
</script>
```

4. LIFECYCLE HOOKS

Composition API Lifecycle

vue

```
<script setup>
import {
  ref,
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted
} from 'vue';

const data = ref(null);
let intervalId = null;

// Antes do componente ser montado
onBeforeMount(() => {
  console.log('Before mount');
});

// ✓ Depois do componente ser montado (DOM disponível)
// USAR PARA: fetch inicial, iniciar timers, setup de libraries
onMounted(() => {
  console.log('Component mounted');

  // Fetch inicial de dados
  fetchData();

  // Iniciar interval
  intervalId = setInterval(() => {
    fetchData();
  }, 30000); // A cada 30 segundos
});

// Antes de atualizar o DOM
onBeforeUpdate(() => {
  console.log('Before update');
});

// Depois de atualizar o DOM
onUpdated(() => {
  console.log('DOM updated');
});

// Antes de desmontar
onBeforeUnmount(() => {
  console.log('Before unmount');
});
```

```
// ✓ Quando componente é removido
// ⚠️ ESSENCIAL para cleanup (evitar memory leaks!)
onUnmounted() => {
  console.log('Component unmounted');

  // ✓ IMPORTANTE: Limpar intervals/timeouts
  if (intervalId) {
    clearInterval(intervalId);
  }

  // Limpar event listeners, etc.
});

async function fetchData() {
  const response = await fetch('/api/data');
  data.value = await response.json();
}
</script>
```

Options API Lifecycle (equivalente)

vue

```
<script>
export default {
  data() {
    return {
      data: null,
      intervalId: null
    };
  },

  beforeMount() {
    console.log('Before mount');
  },

  mounted() {
    console.log('Component mounted');
    this.fetchData();
    this.intervalId = setInterval(this.fetchData, 30000);
  },

  beforeUpdate() {
    console.log('Before update');
  },

  updated() {
    console.log('DOM updated');
  },

  beforeUnmount() {
    console.log('Before unmount');
  },

  unmounted() {
    console.log('Component unmounted');
    if (this.intervalId) {
      clearInterval(this.intervalId);
    }
  },

  methods: {
    async fetchData() {
      const response = await fetch('/api/data');
      this.data = await response.json();
    }
  }
}
</script>
```

5. COMUNICAÇÃO ENTRE COMPONENTES

Props - PAI → FILHO

vue

```
<!-- ChildComponent.vue -->
<template>
  <div>
    <h2>{{ title }}</h2>
    <p>Count: {{ count }}</p>
    <p>User: {{ user.name }}</p>
  </div>
</template>

<script setup>
// ✓ CORRETO - Composition API
const props = defineProps({
  title: {
    type: String,
    required: true
  },
  count: {
    type: Number,
    default: 0
  },
  user: {
    type: Object,
    required: true
  },
  items: {
    type: Array,
    default: () => []
  }
});

// ⚠ NUNCA modificar props diretamente!
// props.count++; // ✗ ERRADO!

// ✓ Se precisar modificar, criar ref local
import { ref } from 'vue';
const localCount = ref(props.count);
</script>

<!-- Alternativa: sintaxe simples -->
<script setup>
// Array simples (menos validação)
const props = defineProps(['title', 'count', 'user']);
</script>
```

vue

```
<!-- ParentComponent.vue -->
<template>
  <ChildComponent
    :title="pageTitle"
    :count="counter"
    :user="currentUser"
    :items="itemList"
  />
</template>

<script setup>
import { ref } from 'vue';
import ChildComponent from './ChildComponent.vue';

const pageTitle = ref('My Title');
const counter = ref(10);
const currentUser = ref({ name: 'João', age: 25 });
const itemList = ref(['a', 'b', 'c']);
</script>
```

Emits - FILHO → PAI

vue

```
<!-- ChildComponent.vue -->
<template>
  <div>
    <p>{{ message }}</p>
    <button @click="handleClick">Send Update</button>
    <button @click="handleDelete">Delete</button>
  </div>
</template>

<script setup>
// ✓ CORRETO - Declarar eventos que o componente emite
const props = defineProps(['message']);
const emit = defineEmits(['update', 'delete']);

function handleClick() {
  // Emitir evento com payload
  emit('update', props.message + ' updated');
}

function handleDelete() {
  // Emitir evento sem payload
  emit('delete');
}
</script>
```

vue

```
<!-- ParentComponent.vue -->
<template>
  <ChildComponent
    :message="text"
    @update="handleUpdate"
    @delete="handleDelete"
  />
</template>

<script setup>
import { ref } from 'vue';
import ChildComponent from './ChildComponent.vue';

const text = ref('Hello');

function handleUpdate(newValue) {
  text.value = newValue;
  console.log('Updated:', newValue);
}

function handleDelete() {
  console.log('Delete requested');
}
</script>
```

v-model em Componentes Personalizados

vue

```
<!-- CustomInput.vue -->
<template>
  <input
    :value="modelValue"
    @input="$emit('update:modelValue', $event.target.value)"
    class="custom-input"
  />
</template>

<script setup>
// v-model = :modelValue + @update:modelValue
defineProps(['modelValue']);
defineEmits(['update:modelValue']);
</script>
```

vue

```
<!-- ParentComponent.vue -->
<template>
  <!-- v-model no componente personalizado -->
  <CustomInput v-model="text" />

  <!-- É equivalente a: -->
  <CustomInput
    :modelValue="text"
    @update:modelValue="text = $event"
  />

  <p>Text: {{ text }}</p>
</template>

<script setup>
import { ref } from 'vue';
import CustomInput from './CustomInput.vue';

const text = ref("");
</script>
```

Múltiplos v-models

vue

```
<!-- UserForm.vue -->
<template>
  <div>
    <input
      :value="firstName"
      @input="$emit('update:firstName', $event.target.value)"
    />
    <input
      :value="lastName"
      @input="$emit('update:lastName', $event.target.value)"
    />
  </div>
</template>

<script setup>
defineProps(['firstName', 'lastName']);
defineEmits(['update:firstName', 'update:lastName']);
</script>
```

vue

```
<!-- Parent.vue -->
<template>
  <UserForm
    v-model:firstName="user.firstName"
    v-model:lastName="user.lastName"
  />
</template>
```

6. PINIA STORES

Template Básico de Store

javascript

// stores/todoStore.js

import { defineStore } from 'pinia';

export const useTodoStore = defineStore('todoStore', {

// ✓ STATE - SEMPRE função que retorna objeto

state: () => ({

todos: [],

filter: 'all',

loading: false

}),

// ✓ GETTERS - Propriedades calculadas (computed)

// NÃO modificam estado, apenas retornam valores derivados

getters: {

// Recebe state como parâmetro

completedTodos: (state) => {

return state.todos.filter(todo => todo.done);

},

activeTodos: (state) => {

return state.todos.filter(todo => !todo.done);

},

todoCount: (state) => state.todos.length,

// Getter que acede a outro getter - usa 'this'

completedCount() {

return this.completedTodos.length;

},

// Getter com parâmetro (retorna função)

getTodoById: (state) => {

return (id) => state.todos.find(todo => todo.id === id);

}

},

// ✓ ACTIONS - Métodos que modificam estado

// Podem ser síncronos ou assíncronos

// Usam 'this' para aceder ao estado

actions: {

// Síncrona

addTodo(text) {

const newTodo = {

id: Date.now(),

text: text,

done: false

```

    };

    this.todos.push(newTodo);
  },

  removeTodo(id) {
    this.todos = this.todos.filter(todo => todo.id !== id);
  },

  toggleTodo(id) {
    const todo = this.todos.find(t => t.id === id);
    if (todo) {
      todo.done = !todo.done;
    }
  },

  setFilter(filter) {
    this.filter = filter;
  },

  // ✓ Assíncrona - pode usar async/await
  async fetchTodos() {
    this.loading = true;
    try {
      const response = await fetch('/api/todos');
      const data = await response.json();
      this.todos = data;
    } catch (error) {
      console.error('Error fetching todos:', error);
    } finally {
      this.loading = false;
    }
  }
});

```

✗ ERROS COMUNS em Pinia

javascript

// ✗ ERRADO - data: function() é de Options API components

```
export const useStore = defineStore('store', {  
  data: function() { // ✗ ERRADO!  
    return { items: [] };  
  }  
});
```

// ✓ CORRETO - state: () => ({ })

```
export const useStore = defineStore('store', {  
  state: () => ({  
    items: []  
  })  
});
```

// ✗ ERRADO - methods não existe em Pinia

```
export const useStore = defineStore('store', {  
  methods: { // ✗ ERRADO!  
    addItem() {}  
  }  
});
```

// ✓ CORRETO - actions

```
export const useStore = defineStore('store', {  
  actions: {  
    addItem() {}  
  }  
});
```

// ✗ ERRADO - getters modificam estado

```
export const useStore = defineStore('store', {  
  getters: {  
    addToCount: (state) => {  
      state.count++; // ✗ ERRADO!  
    }  
  }  
});
```

// ✓ CORRETO - getters apenas retornam valores

```
export const useStore = defineStore('store', {  
  getters: {  
    doubledCount: (state) => state.count * 2  
  }  
});
```

// ✗ ERRADO - cronoName = this.name

```
export const useStore = defineStore('store', {  
  actions: {  
    // ...  
  }  
});
```

```
    setName(name) {  
      cronoName = this.name; // ✗ ERRADO!  
    }  
  }  
});  
  
// ✓ CORRETO - this.cronoName = name  
export const useStore = defineStore('store', {  
  actions: {  
    setName(name) {  
      this.cronoName = name;  
    }  
  }  
});
```

Usar Store num Componente

vue

```
<template>
  <div>
    <!-- Aceder a state -->
    <p>Total: {{ store.todoCount }}</p>

    <!-- Aceder a getters -->
    <p>Completed: {{ store.completedCount }}</p>

    <!-- Lista de todos -->
    <ul>
      <li v-for="todo in store.todos" :key="todo.id">
        {{ todo.text }}
        <button @click="store.toggleTodo(todo.id)">Toggle</button>
        <button @click="store.removeTodo(todo.id)">Remove</button>
      </li>
    </ul>

    <!-- Adicionar novo todo -->
    <input v-model="newTodo" @keyup.enter="addTodo" />
    <button @click="addTodo">Add</button>
  </div>
</template>
```

```
<script setup>
import { ref } from 'vue';
import { useTodoStore } from '@stores/todoStore';

// Inicializar store
const store = useTodoStore();

const newTodo = ref("");

function addTodo() {
  if (newTodo.value.trim()) {
    // Chamar action
    store.addTodo(newTodo.value);
    newTodo.value = "";
  }
}

// Fetch inicial
store.fetchTodos();
</script>
```

Store Completa - Exemplo Real

javascript

// stores/userStore.js

import { defineStore } from 'pinia';

```
export const useUserStore = defineStore('userStore', {
  state: () => ({
    user: null,
    isAuthenticated: false,
    loading: false,
    error: null
  }),
```

```
  getters: {
    userName: (state) => state.user?.name || 'Guest',
    isAdmin: (state) => state.user?.role === 'admin',
    userInitials() {
      if (!this.user?.name) return '';
      return this.user.name
        .split(' ')
        .map(n => n[0])
        .join("")
        .toUpperCase();
    }
  },
```

```
  actions: {
    async login(email, password) {
      this.loading = true;
      this.error = null;

      try {
        const response = await fetch('/api/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password })
        });

        if (!response.ok) {
          throw new Error('Login failed');
        }

        const data = await response.json();
        this.user = data.user;
        this.isAuthenticated = true;

        // Guardar token
        localStorage.setItem('token', data.token);
      } catch (error) {
        this.error = error.message;
      }
    }
  }
});
```

```
    } catch (error) {  
      this.error = error.message;  
      this.isAuthenticated = false;  
    } finally {  
      this.loading = false;  
    }  
  },  
  
  logout() {  
    this.user = null;  
    this.isAuthenticated = false;  
    localStorage.removeItem('token');  
  },  
  
  updateProfile(updates) {  
    if (this.user) {  
      this.user = { ...this.user, ...updates };  
    }  
  }  
}  
});
```

7. VUE ROUTER

Configuração Básica

javascript

// router/index.js

```
import { createRouter, createWebHistory } from 'vue-router';
import Home from '@views/Home.vue';
import About from '@views/About.vue';
import UserProfile from '@views/UserProfile.vue';
import NotFound from '@views/NotFound.vue';
```

```
const routes = [
  {
    path: '/',
    name: 'home',
    component: Home
  },
  {
    path: '/about',
    name: 'about',
    component: About
  },
  {
    // ✓ Rota com parâmetro dinâmico
    path: '/user/:id',
    name: 'user',
    component: UserProfile,
    props: true // ✓ Passa :id como prop ao componente
  },
  {
    // ✓ Rotas aninhadas (nested routes)
    path: '/admin',
    component: () => import('@layouts/AdminLayout.vue'),
    children: [
      {
        path: '', // /admin
        component: () => import('@views/admin/Dashboard.vue')
      },
      {
        path: 'users', // /admin/users
        component: () => import('@views/admin/Users.vue')
      },
      {
        path: 'settings', // /admin/settings
        component: () => import('@views/admin/Settings.vue')
      }
    ]
  },
  {
    // ✓ Lazy loading - carrega só quando necessário
    path: '/dashboard',
    component: () => import('@views/Dashboard.vue')
  }
];
```

```

    component: () => import('@views/Dashboard.vue')
  },
  {
    // ✓ 404 - catch-all (sempre no fim!)
    path: '/*',
    name: 'not-found',
    component: NotFound
  }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

// Navigation guards (opcional)
router.beforeEach((to, from, next) => {
  // Verificar autenticação, etc.
  console.log(`Navigating from ${from.path} to ${to.path}`);
  next();
});

export default router;

```

Usar Router no main.js

```

javascript

// main.js
import { createApp } from 'vue';
import { createPinia } from 'pinia';
import App from './App.vue';
import router from './router';

const app = createApp(App);
const pinia = createPinia();

app.use(pinia);
app.use(router);
app.mount('#app');

```

Navegação no Template

vue

```
<template>
  <nav>
    <!-- ✓ Links básicos -->
    <router-link to="/">Home</router-link>
    <router-link to="/about">About</router-link>

    <!-- ✓ Link com rota nomeada -->
    <router-link :to="{ name: 'user', params: { id: userId }}">
      My Profile
    </router-link>

    <!-- ✓ Link com query params -->
    <router-link :to="{ path: '/search', query: { q: 'vue' }}">
      Search
    </router-link>

    <!-- ✓ Active class personalizada -->
    <router-link to="/about" active-class="selected">
      About
    </router-link>

    <!-- ✓ Exact active class (apenas se rota exata) -->
    <router-link to="/" exact-active-class="active-home">
      Home
    </router-link>
  </nav>

  <!-- ✓ Onde o componente da rota é renderizado -->
  <router-view />

  <!-- Com transições -->
  <router-view v-slot="{ Component }">
    <transition name="fade">
      <component :is="Component" />
    </transition>
  </router-view>
</template>

<script setup>
import { ref } from 'vue';

const userId = ref(42);
</script>

<style>
.selected {
  color: #3498db;
  font-weight: bold;
}
```



```
font-weight: bold;
}

.fade-enter-active, .fade-leave-active {
  transition: opacity 0.3s;
}
.fade-enter-from, .fade-leave-to {
  opacity: 0;
}
</style>
```

Navegação Programática

vue

```
<script setup>
import { useRouter, useRoute } from 'vue-router';

const router = useRouter();
const route = useRoute();

// ✓ Navegar para rota
function goToAbout() {
  router.push('/about');
}

// ✓ Navegar com rota nomeada
function goToUser(userId) {
  router.push({
    name: 'user',
    params: { id: userId }
  });
}

// ✓ Navegar com query params
function goToSearch(query) {
  router.push({
    path: '/search',
    query: { q: query }
  });
}

// ✓ Substituir entrada no histórico (não adiciona)
function replaceRoute() {
  router.replace('/dashboard');
}

// ✓ Voltar
function goBack() {
  router.back();
}

// ✓ Avançar
function goForward() {
  router.forward();
}

// ✓ Ir N páginas para trás/frente
function go(n) {
  router.go(n); // -1 = voltar, 1 = avançar
}
```

```
// ✓ Aceder a informação da rota atual
console.log(route.path);    // '/user/42'
console.log(route.params);  // { id: '42' }
console.log(route.query);   // { search: 'vue' }
console.log(route.name);    // 'user'
</script>
```

Receber Parâmetros no Componente

```
vue

<!-- UserProfile.vue -->
<template>
  <div>
    <h1>User Profile</h1>
    <p>User ID: {{ id }}</p>
    <p>From route: {{ $route.params.id }}</p>
  </div>
</template>

<script setup>
import { useRoute } from 'vue-router';

// ✓ Método 1: Como prop (requer props: true no router)
const props = defineProps(['id']);

// ✓ Método 2: Via useRoute()
const route = useRoute();
console.log(route.params.id);
</script>
```

8. FETCH API E ASYNC

Padrão Básico com .then()

javascript

// ✓ CORRETO - fetch retorna Promise

```
fetch('http://api.com/items')
```

```
.then(response => response.json()) // .json() também retorna Promise!
```

```
.then(data => {
```

```
  this.list = data; // ✓ Guardar em propriedade reativa
```

```
})
```

```
.catch(error => {
```

```
  console.error('Error:', error);
```

```
});
```

// ✗ ERRADO - guarda Promise, não os dados

```
this.list = fetch('http://api.com/items'); // ✗ ERRADO!
```

```
this.list = fetch('...').then(r => r.json()); // ✗ ERRADO!
```

Padrão com async/await (RECOMENDADO)

vue

```
<script setup>
import { ref, onMounted } from 'vue';

const users = ref([]);
const loading = ref(false);
const error = ref(null);

// ✓ CORRETO - async/await
async function fetchUsers() {
  loading.value = true;
  error.value = null;

  try {
    const response = await fetch('/api/users');

    // ✓ Verificar se resposta é OK (status 200-299)
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    users.value = data;

  } catch (err) {
    error.value = err.message;
    console.error('Fetch error:', err);

  } finally {
    loading.value = false;
  }
}
```

```
// Fetch inicial quando componente monta
onMounted(() => {
  fetchUsers();
});
</script>
```

```
<template>
  <div>
    <button @click="fetchUsers">Refresh</button>

    <div v-if="loading">Loading...</div>
    <div v-else-if="error">Error: {{ error }}</div>
    <ul v-else>
      <li v-for="user in users" :key="user.id">
        {{ user.name }}
      </li>
    </ul>
  </div>
</template>
```

```
</li>
</ul>
</div>
</template>
```

POST Request

javascript

```
async function createUser(userData) {
  try {
    const response = await fetch('/api/users', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(userData)
    });

    if (!response.ok) {
      throw new Error('Failed to create user');
    }

    const newUser = await response.json();
    return newUser;

  } catch (error) {
    console.error('Error creating user:', error);
    throw error;
  }
}

// Uso:
async function handleSubmit() {
  try {
    const newUser = await createUser({
      name: 'João',
      email: 'joao@example.com'
    });
    console.log('Created:', newUser);
  } catch (error) {
    // Tratar erro
  }
}
```

Fetch numa Store Pinia

javascript

// stores/dataStore.js

import { defineStore } from 'pinia';

export const useDataStore = defineStore('dataStore', {

state: () => ({

items: [],

loading: false,

error: null

}),

actions: {

async fetchItems() {

this.loading = true;

this.error = null;

try {

const response = await fetch('/api/items');

if (!response.ok) {

throw new Error('Network response was not ok');

}

const data = await response.json();

this.items = data;

} catch (error) {

this.error = error.message;

console.error('Error fetching items:', error);

} finally {

this.loading = false;

}

},

async addItem(item) {

try {

const response = await fetch('/api/items', {

method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify(item)

});

const newItem = await response.json();

this.items.push(newItem);

return newItem;

} catch (error) {

```
    console.error('Error adding item:', error);  
    throw error;  
  }  
}  
}  
});
```

9. FORMULÁRIOS E V-MODEL

Formulário Completo

vue

```
<template>
  <form @submit.prevent="handleSubmit">
    <!-- Text input -->
    <div>
      <label for="name">Name:</label>
      <input
        id="name"
        v-model="form.name"
        type="text"
        required
      />
    </div>

    <!-- Email input -->
    <div>
      <label for="email">Email:</label>
      <input
        id="email"
        v-model="form.email"
        type="email"
        required
      />
    </div>

    <!-- Password -->
    <div>
      <label for="password">Password:</label>
      <input
        id="password"
        v-model="form.password"
        type="password"
        minlength="6"
        required
      />
    </div>

    <!-- Number input -->
    <div>
      <label for="age">Age:</label>
      <input
        id="age"
        v-model.number="form.age"
        type="number"
        min="18"
      />
    </div>
  </form>
</template>
```

```
<!-- Checkbox (single)-->
<div>
  <label>
    <input
      v-model="form.acceptTerms"
      type="checkbox"
    />
    Accept terms and conditions
  </label>
</div>
```

```
<!-- Checkbox (multiple) -->
<div>
  <p>Interests:</p>
  <label>
    <input
      v-model="form.interests"
      type="checkbox"
      value="sports"
    />
    Sports
  </label>
  <label>
    <input
      v-model="form.interests"
      type="checkbox"
      value="music"
    />
    Music
  </label>
  <label>
    <input
      v-model="form.interests"
      type="checkbox"
      value="reading"
    />
    Reading
  </label>
</div>
```

```
<!-- Radio buttons -->
<div>
  <p>Gender:</p>
  <label>
    <input
      v-model="form.gender"
      type="radio"
      value="male"
    />
    Male
```

```

</label>
<label>
  <input
    v-model="form.gender"
    type="radio"
    value="female"
  />
  Female
</label>
<label>
  <input
    v-model="form.gender"
    type="radio"
    value="other"
  />
  Other
</label>
</div>

```

```

<!-- Select -->
<div>
  <label for="country">Country:</label>
  <select id="country" v-model="form.country">
    <option value="">Select...</option>
    <option value="pt">Portugal</option>
    <option value="es">Spain</option>
    <option value="fr">France</option>
  </select>
</div>

```

```

<!-- Select múltiplo -->
<div>
  <label for="skills">Skills:</label>
  <select id="skills" v-model="form.skills" multiple>
    <option value="js">JavaScript</option>
    <option value="vue">Vue.js</option>
    <option value="css">CSS</option>
  </select>
</div>

```

```

<!-- Textarea -->
<div>
  <label for="bio">Bio:</label>
  <textarea
    id="bio"
    v-model="form.bio"
    rows="4"
  ></textarea>
</div>

```

```
<!-- Submit -->
<button type="submit">Submit</button>
<button type="button" @click="resetForm">Reset</button>
</form>
```

```
<!-- Preview -->
<div>
  <h3>Form Data:</h3>
  <pre>{{ form }}</pre>
</div>
</template>
```

```
<script setup>
```

```
import { reactive } from 'vue';
```

```
const form = reactive({
  name: "",
  email: "",
  password: "",
  age: null,
  acceptTerms: false,
  interests: [],
  gender: "",
  country: "",
  skills: [],
  bio: ""
});
```

```
function handleSubmit() {
  console.log('Form submitted:', form);
```

```
  // Validação adicional
  if (!form.acceptTerms) {
    alert('Please accept terms');
    return;
  }
```

```
  // Enviar para API
  submitToAPI(form);
}
```

```
function resetForm() {
  Object.assign(form, {
    name: "",
    email: "",
    password: "",
    age: null,
    acceptTerms: false,
    interests: []
```

```
interests: [],
gender: "",
country: "",
skills: [],
bio: ""
});
}

async function submitToAPI(data) {
  try {
    const response = await fetch('/api/submit', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });

    if (response.ok) {
      alert('Success!');
      resetForm();
    }
  } catch (error) {
    console.error('Error:', error);
  }
}
</script>
```

Modificadores de v-model

```
vue

<template>
  <!-- .lazy - atualiza no change, não no input -->
  <input v-model.lazy="text" />

  <!-- .number - converte para número -->
  <input v-model.number="age" type="number" />

  <!-- .trim - remove espaços -->
  <input v-model.trim="username" />

  <!-- Combinar modificadores -->
  <input v-model.trim.lazy="text" />
</template>
```

10. SLOTS

Slot Básico

vue

```
<!-- Card.vue -->
<template>
  <div class="card">
    <slot>
      <!-- ✓ Conteúdo default se nada for passado -->
      Default card content
    </slot>
  </div>
</template>
```

vue

```
<!-- Parent.vue -->
<template>
  <!-- Passar conteúdo para o slot -->
  <Card>
    <h2>My Title</h2>
    <p>My custom content</p>
  </Card>

  <!-- Sem conteúdo - mostra default -->
  <Card />
</template>
```

Named Slots

vue

```
<!-- Card.vue -->
<template>
  <div class="card">
    <header class="card-header">
      <slot name="header">Default Header</slot>
    </header>

    <main class="card-body">
      <!-- Slot sem nome = "default" -->
      <slot>Default Content</slot>
    </main>

    <footer class="card-footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>

<style scoped>
.card {
  border: 1px solid #ddd;
  border-radius: 4px;
}
.card-header {
  background: #f5f5f5;
  padding: 1rem;
}
.card-body {
  padding: 1rem;
}
.card-footer {
  background: #f5f5f5;
  padding: 0.5rem;
}
</style>
```

vue

```
<!-- Parent.vue -->
<template>
  <Card>
    <!-- #header é atalho para v-slot:header -->
    <template #header>
      <h1>My Card Title</h1>
    </template>

    <!-- Conteúdo default (sem template) -->
    <p>This is the main content</p>

    <template #footer>
      <button>Action</button>
    </template>
  </Card>
</template>
```

Scoped Slots

vue

```
<!-- List.vue -->
<template>
  <ul>
    <li v-for="item in items" :key="item.id">
      <!-- ✓ Passar dados para o slot -->
      <slot :item="item" :index="item.id">
        {{ item.name }}
      </slot>
    </li>
  </ul>
</template>

<script setup>
defineProps(['items']);
</script>
```


vue

```
<!-- Parent.vue -->
<template>
  <!-- Receber dados do slot -->
  <List :items="users">
    <template #default="{ item, index }">
      <strong>{{ index }}:</strong> {{ item.name }} - {{ item.email }}
    </template>
  </List>

  <!-- Sintaxe alternativa -->
  <List :items="users" v-slot="{ item }">
    <strong>{{ item.name }}</strong>
  </List>
</template>

<script setup>
import { ref } from 'vue';

const users = ref([
  { id: 1, name: 'João', email: 'joao@example.com' },
  { id: 2, name: 'Maria', email: 'maria@example.com' }
]);
</script>
```

11. WEB COMPONENTS

Custom Element Básico

javascript

// my-counter.js

// ✓ Classe extends HTMLElement

class MyCounter extends HTMLElement {

constructor() {

super();

this.count = 0;

// ✓ Criar Shadow DOM (encapsula estilos)

this.attachShadow({ mode: 'open' });

this.render();

}

// ✓ Chamado quando elemento é adicionado ao DOM

connectedCallback() {

const button = this.shadowRoot.querySelector('button');

button.addEventListener('click', () => this.increment());

}

// ✓ Chamado quando elemento é removido

disconnectedCallback() {

// Cleanup: remover event listeners, etc.

}

// ✓ Atributos que serão observados

static get observedAttributes() {

return ['initial', 'step'];

}

// ✓ Chamado quando atributos mudam

attributeChangedCallback(name, oldValue, newValue) {

if (name === 'initial') {

this.count = parseInt(newValue) || 0;

this.render();

}

if (name === 'step') {

this.step = parseInt(newValue) || 1;

}

}

increment() {

this.count += (this.step || 1);

this.render();

// ✓ Emitir evento personalizado

this.dispatchEvent(new CustomEvent('count-changed', {

```

    detail: { count: this.count },
    bubbles: true
  });
}

render() {
  // ✓ Estilos encapsulados no Shadow DOM
  this.shadowRoot.innerHTML = `
    <style>
      button {
        padding: 10px 20px;
        font-size: 16px;
        background: #3498db;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
      }
      button:hover {
        background: #2980b9;
      }
      .count {
        font-size: 24px;
        margin: 10px 0;
      }
    </style>
    <div>
      <div class="count">Count: ${this.count}</div>
      <button>Increment</button>
    </div>
  `;
}

// ✓ Registrar elemento - nome DEVE ter hífen!
customElements.define('my-counter', MyCounter);

```

Usar o Web Component

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Web Component Example</title>
  <script src="my-counter.js"></script>
</head>
<body>
  <!-- ✓ Usar o custom element -->
  <my-counter></my-counter>

  <!-- Com atributos -->
  <my-counter initial="10" step="2"></my-counter>

  <!-- Escutar eventos -->
  <my-counter id="counter"></my-counter>

  <script>
    const counter = document.getElementById('counter');
    counter.addEventListener('count-changed', (e) => {
      console.log('New count:', e.detail.count);
    });
  </script>
</body>
</html>
```

Web Component com Slots

javascript

```
class MyCard extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });

    this.shadowRoot.innerHTML = `
      <style>
        .card {
          border: 1px solid #ddd;
          border-radius: 8px;
          padding: 1rem;
        }
        .header {
          font-weight: bold;
          margin-bottom: 1rem;
        }
      </style>
      <div class="card">
        <div class="header">
          <!-- ✓ Slot nomeado -->
          <slot name="header">Default Header</slot>
        </div>
        <div class="content">
          <!-- ✓ Slot default -->
          <slot></slot>
        </div>
      </div>
    `;
  }
}
```

customElements.define('my-card', MyCard);

html

```
<!-- Uso com slots -->
<my-card>
  <span slot="header">Card Title</span>
  <p>This is the card content</p>
</my-card>
```

Pontos-chave Web Components

javascript

// ✓ CORRETO

```
class MyElement extends HTMLElement { }  
customElements.define('my-element', MyElement);
```

// ✗ ERRADO - nome sem hífen

```
customElements.define('myelement', MyElement); // ✗ ERRO!
```

// ✓ Callbacks importantes

```
class MyElement extends HTMLElement {  
  connectedCallback() {  
    // Elemento adicionado ao DOM  
  }  
}
```

```
  disconnectedCallback() {  
    // Elemento removido - CLEANUP aqui!  
  }  
}
```

```
  adoptedCallback() {  
    // Movido para novo document  
  }  
}
```

```
  attributeChangedCallback(name, oldVal, newVal) {  
    // Atributo mudou  
  }  
}
```

// ✓ NÃO requer framework - são nativos do browser!

// ✓ Podem ser usados em qualquer página HTML

// ✓ Encapsulam estilos e comportamento

REFERÊNCIA RÁPIDA

✗ Erros Comuns

javascript

// Pinia

✗ `data: function() { }` → ✓ `state: () => ({})`
✗ `methods: { }` → ✓ `actions: { }`
✗ `getter modifica estado` → ✓ `getter apenas retorna`
✗ `cronoName = this.name` → ✓ `this.cronoName = name`

// Vue

✗ `reactive(0)` → ✓ `ref(0)`
✗ `{{ count.value }}` template → ✓ `{{ count }}`
✗ `:key="index"` lista dinâmica → ✓ `:key="item.id"`

// Fetch

✗ `this.list = fetch(...)` → ✓ `fetch(...).then(r => r.json()).then(d => this.list = d)`

// Lifecycle

✗ esquecer `clearInterval` → ✓ sempre limpar no `onUnmounted`

✓ Boas Práticas

javascript

// Reatividade

- `ref()` para **primitivos** (string, number, boolean)
- `reactive()` para objetos e arrays
- `computed()` para valores **derivados** (mais eficiente)

// Componentes

- Props: pai → **filho** (read-only!)
- Emits: filho → **pai** (comunicação)
- v-model: two-way binding

// Store

- State: dados reativos
- Getters: valores **derivados** (computed)
- Actions: modificar **estado** (podem ser async)

// Lifecycle

- `onMounted`: fetch inicial, setup
- `onUnmounted`: **SEMPRE** fazer cleanup!

FIM DO DOCUMENTO